Figure 24.7 If you're logged in but not authorized to edit a recipe, you'll be redirected to an "Access Denied" page. If you're not logged in, you'll be redirected to the Login page.

By using resource-based authorization, you're able to enact more fine-grained authorization requirements that you can apply at the level of an individual document or resource. Instead of being able to authorize only that a user can edit *any* recipe, you can authorize whether a user can edit *this* recipe.

All the authorization techniques you've seen so far have focused on server-side checks. Both the `[Authorize]` attribute and resource-based authorization approaches focus on stopping users from executing a protected endpoint on the server. This is important from a security point of view, but there's another aspect you should consider: the user experience when they don't have permission.

You've protected the code executing on the server, but arguably the Edit button should never have been visible to the user if they weren't going to be allowed to edit the recipe! In the next section we'll look at how you can conditionally hide the Edit button by using resource-based authorization in your view models.

Resource-based authorization versus business-logic checks

The value proposition of using the ASP.NET Core framework's resource-based authorization approach isn't always clear compared with using simple, manual, business-logic based checks (as in listing 24.13). Using `IAuthorizationService` and the authorization infrastructure adds an explicit dependency on the ASP.NET Core framework that you may not want to use if you're performing authorization checks in your domain model services.